



南京林业大学
NANJING FORESTRY UNIVERSITY

FeHALS：基于 Web 的 3D 可视化航路规划与激光仿真系统

课程名称： “智能 +”GIS 设计与开发

专业： 测绘工程

组别： 7 组

年级： 2023 级

小组成员：	王孜悠	2310604116
	梅康凯	2310604109
	唐如意	2310604115
	杨杰瑞	2310604122
	温璞	2310604117
	刘继明	2110604112
	李英杰	2310604106
	杜仲宇	2310604103
	施祺	2310604113
	向宇鸣	2310604118

指导老师： 隋铭明 那嘉明 陈动 朱杰

批改日期：

提交日期： 2026 年 9 月 16 日

Graphical Abstract

FeHALS: 基于 Web 的 3D 可视化航路规划与激光仿真系统

王孜悠, 温璞, 梅康凯, 唐如意, 杨杰瑞, 杜仲宇, 李英杰, 施祺, 刘继明, 向宇鸣, 隋铭明, 那嘉明, 陈动, 朱杰

FeHALS



Highlights

FeHALS：基于 Web 的 3D 可视化航路规划与激光仿真系统

王孜悠, 温璞, 梅康凯, 唐如意, 杨杰瑞, 杜仲宇, 李英杰, 施祺, 刘继明, 向宇鸣, 隋铭明, 那嘉明, 陈动, 朱杰

- 设计并实现了基于 Web 的 HELIOS++ 仿真参数图形化配置界面，自动生成 XML 配置文件与航迹文件。
- 构建了从场景构建、航路设计、仿真执行到结果渲染的完整闭环工作流，实现了激光仿真全流程的集成化与可视化。
- 系统基于开源技术栈 (Vue 3 + Three.js + FastAPI + HELIOS++), 前后端解耦，具有良好的可扩展性与跨平台访问能力。
- 使用 git+github 的本地 + 云端组合统筹开发流程，深度落实了现代化软件开发协作机制。

FeHALS: 基于 Web 的 3D 可视化航路规划与激光仿真系统

王 孜悠^a, 温 璞^a, 梅 康凯^a, 唐 如意^a, 杨 杰瑞^a, 杜 仲宇^a, 李 英杰^a, 施 祺^a, 刘 继明^a, 向 宇鸣^a, 隋 铭明^{a,1}, 那 嘉明^{a,1}, 陈 动^{a,1} and 朱 杰^{a,1}

^a College of Civil Engineering, Nanjing Forestry University, Longpan Rd., Nanjing, 210037, China

ARTICLE INFO

Keywords:

ALS
航路规划
三维可视化
B/S 架构
HELIOS++

摘要

三维激光扫描仿真技术在地理信息、遥感与测绘领域的应用日益广泛。HELIOS++ 作为一款开源的激光雷达仿真框架,能够高保真地模拟机载、地面与无人机等平台的扫描过程,但其原生交互方式依赖命令行与 XML 配置文件,缺乏直观的三维可视化航路规划能力,限制了其在复杂场景下的高效使用。本文提出并实现了一个基于 Web 的 FeHALS (Frontend of HELIOS++ for Airborne Laser Scanning, 最初面向机载激光扫描,现已扩展支持车载与地基平台) 系统。系统采用浏览器/服务器 (B/S) 架构,前端基于 Vue 3 与 Three.js 构建三维交互场景,后端基于 FastAPI 提供仿真服务并封装 HELIOS++ 引擎调用。系统支持导入多种三维模型,用户可在浏览器三维场景中通过鼠标交互设定航点并自动生成符合 HELIOS++ 格式的航迹文件;同时提供图形化界面配置扫描平台与传感器参数,一键调用仿真引擎并通过 WebSocket 实时推送运行日志;仿真生成的点云可自动加载至三维场景,并支持按高度、强度着色以及点尺寸与透明度的调节。在项目管理方面,本文作者作为仓库唯一的 CODEOWNER,全面负责系统架构设计、技术路线决策、核心代码实现、文档编写与团队协作的统筹管理,通过 GitHub 的 Pull Request 与分支保护机制确保代码质量与协作效率。该系统有效降低了 HELIOS++ 的使用门槛,为航摄方案设计、传感器参数优化及点云数据模拟提供了便捷、高效的可视化工具。本文详细阐述了系统的架构设计、核心功能模块与技术实现路径,并通过典型场景验证了系统的可行性与实用性。源代码已公开至<https://github.com/WZYivan/FeHALS>。

目录

1	引言	2
2	相关技术	3
2.1	HELIOS++ 仿真框架	3
2.2	Web 三维可视化与 Three.js	3
2.3	FastAPI 与后端异步任务	3
3	系统需求分析与设计	4
3.1	用户需求分析	4
3.2	系统总体架构	4
3.3	功能优先级划分	4
4	系统设计与实现	5
4.1	系统模块划分	5
4.2	三维场景渲染与航点交互	5
4.3	前端组件与状态管理	5
4.4	后端服务与接口	6
4.5	航迹与配置文件生成	7

Github User Name: WZYivan (王. 孜悠); wenpu123@users.noreply.github.com (温. 璞); zhangqilinghy (梅. 康凯); Try6743 (唐. 如意); 19951391030yjr-sudo (杨. 杰瑞); 2309175724@qq.com (杜. 仲宇); 1589428766@qq.com (李. 英杰); 69901001+nidie1@users.noreply.github.com (施. 祺); 2360122668@qq.com (刘. 继明)

学号: 2310604116 (王. 孜悠); 2310604117 (温. 璞); 2310604109 (梅. 康凯); 2310604115 (唐. 如意); 2310604122 (杨. 杰瑞); 2310604103 (杜. 仲宇); 2310604106 (李. 英杰); 2310604113 (施. 祺); 2110604112 (刘. 继明); 2310604118 (向. 宇鸣)

¹指导老师

4.6 HELIOS++ 调用与点云渲染	7
5 结论与展望	7
A 详细的贡献	9
B FeHALS 示例 workflow	10

1. 引言

激光雷达 (Light Detection and Ranging, LiDAR) 通过发射激光脉冲并测量其回波时间与强度, 能够快速获取地物表面的高精度三维点云, 已成为航空摄影测量、林业资源调查、城市三维建模与地形测绘等领域的核心数据获取手段之一 (Yao et al., 2022)。机载激光扫描 (Airborne Laser Scanning, ALS) 将激光扫描仪搭载于飞行平台, 沿预定航线对地面进行扫描, 其点云质量与航高、航速、扫描频率、脉冲频率及航线布设等参数密切相关。然而, 实际航摄作业受空域、成本与环境等因素制约, 方案设计与参数验证往往缺乏充分的试验条件, 因此通过计算机仿真对扫描过程进行预演、评估与优化具有重要的工程意义。

HELIOS++ (Winiwarter et al., 2022) 是一款开源的激光雷达仿真框架, 基于光线追踪算法能够高保真地模拟机载、地面、车载与无人机等平台的扫描过程, 并支持全波形输出。但其原生使用方式依赖命令行与手写 XML 配置文件, 用户需要熟练掌握场景 (scene)、扫描任务 (survey)、平台 (platform) 与扫描器 (scanner) 等配置文件的语法, 且无法直观地在三维场景中规划航线, 学习与使用门槛较高。

HELIOS++ 官方团队已提供了一个基于 QGIS 的图形化插件 AEOS (3D Geo Research Group, Heidelberg University, 2022), 将 HELIOS++ 的仿真配置与执行功能集成到 QGIS 桌面环境中。AEOS 作为 QGIS 插件, 充分利用了 QGIS 的三维地图渲染与地理空间数据管理能力, 适合需要与 GIS 工作流深度结合的用户。然而, AEOS 的使用依赖于 QGIS 桌面环境的完整安装与配置, 对于仅需快速进行航路规划与仿真参数调试的用户而言, 启动和配置成本较高, 且无法在浏览器或轻量级环境中直接使用。

针对上述问题, 本文设计并实现了 FeHALS (Frontend of HELIOS++ for Airborne Laser Scanning, 最初面向机载激光扫描, 现已扩展支持车载与地基平台), 一个基于 Web 的三维可视化航路规划与激光仿真系统, 其 Web 前端如图 1 所示。与 AEOS 的 QGIS 集成路线不同, FeHALS 采用浏览器/服务器 (B/S) 架构: 前端基于 Vue 3 (You, 2014) 与 Three.js (Cabello, 2010) 构建三维交互场景, 提供模型加载、交互式航点编辑、参数配置与点云渲染等功能; 后端基于 FastAPI (Ramírez, 2018) 实现, 负责航迹与 XML 配置文件的生成, 并通过子进程调用 HELIOS++ 引擎执行仿真, 经 WebSocket (Fette and Melnikov, 2011) 将运行日志实时推送到前端。用户通过浏览器即可完成从场景构建、航路设计、仿真执行到结果可视化的全流程操作, 无需安装任何客户端软件。

在项目开发过程中, 我们全程采用 Git 与 GitHub 作为版本控制与协作平台。所有代码变更均通过分支 (branch) 提交, 并经由 Pull Request (PR) 发起合并请求, 由仓库唯一的 CODEOWNER (王孜悠, WZYivan) 进行代码审查 (review) 后方可合并至主分支。这一机制借助 GitHub 的分支保护规则与 CODEOWNERS 文件实现, 确保了每次合入的代码都经过审核, 历史记录清晰可溯。同时, 团队成员通过 Fork + PR 的工作流贡献代码, 既保证了主仓库的稳定性, 也降低了协作门槛。CODEOWNER 不仅负责代码审查与合并, 还承担了系统架构设计、技术路线决策、核心代码实现 (包括三维场景渲染、航点交互、航迹生成、HELIOS++ 集成、点云解析等关键模块)、文档编写与项目管理统筹等角色。从技术选型 (Vue 3 + Three.js + FastAPI, 摒弃 QGIS 路线) 到开发规范 (Git 工作流、分支策略、CHANGELOG 与 TODO 维护), 均由 CODEOWNER 主导推进。这种本地开发与云端协作相结合的现代化软件开发实践, 不仅提升了团队的协作效率, 也充分体现了软件工程最佳规范在本项目中的落地。详细的贡献可见 Appendix A 或直接访问 FeHALS github Main Repo。

本文的主要贡献包括: (1) 提出一种面向 HELIOS++ 的 Web 端三维航路规划交互方案, 通过鼠标点击与拖拽即可在三维场景中完成航点设定与航线编辑; (2) 实现仿真参数的图形化配置与 XML、航迹文件的自动生成, 屏蔽了 HELIOS++ 底层配置文件的复杂性; (3) 实现仿真执行过程的日志实时推送与点云结果的可视化渲染, 形成完整的闭环工作流。

本文其余部分组织如下:Section 2介绍相关技术;Section 3进行系统需求分析与总体设计;第Section 4阐述系统各模块的设计与实现; Section 5给出结论与展望。

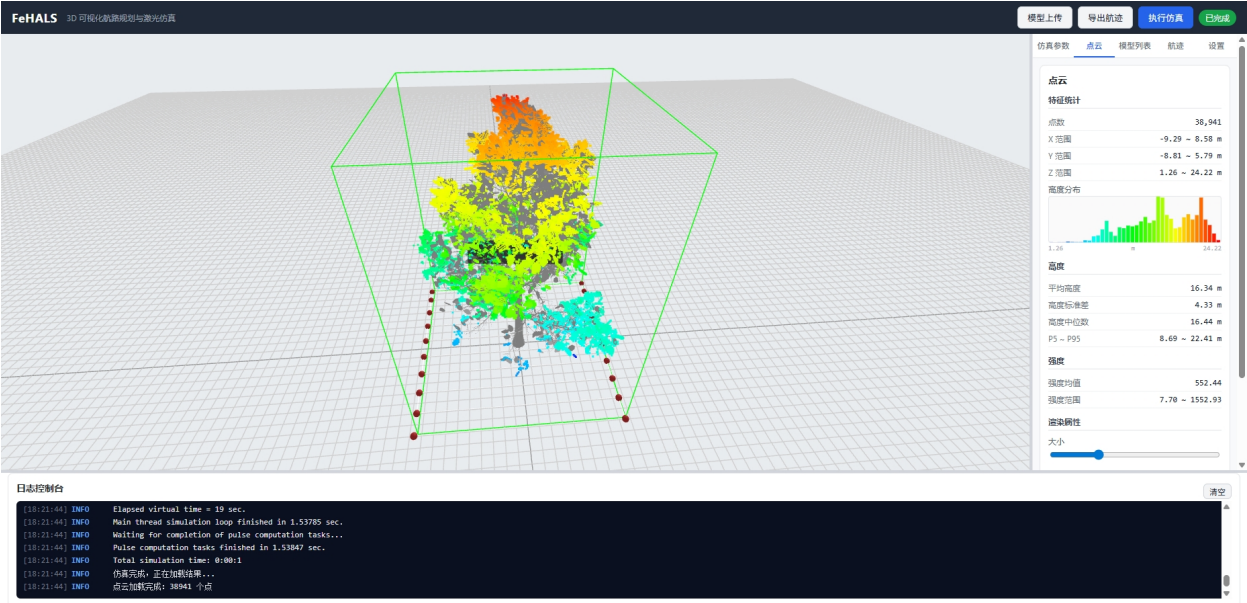


Fig. 1: FeHALS 的 Web 前端与仿真结果

2. 相关技术

2.1. HELIOS++ 仿真框架

HELIOS++ 是一款基于 C++ 的开源激光雷达仿真框架，采用光线追踪技术对激光脉冲在三维场景中的传播过程进行建模，能够模拟机载、地面、车载与无人机等多种平台，支持全波形 (full-waveform) 输出与 LAS、LAZ、XYZ 等多种点云格式。其仿真配置以 XML 文件组织：场景 (scene) 文件定义参与扫描的三维地物模型；平台 (platform) 与扫描器 (scanner) 目录文件定义飞行平台与扫描仪的物理参数；扫描任务 (survey) 文件将场景、平台、扫描器与航迹 (trajectory) 关联起来，并设定脉冲频率、扫描频率与扫描角等核心参数。HELIOS++ 提供命令行接口，通过指定 survey 文件、资源目录 (-assets) 与输出目录 (-output) 即可启动仿真。由于配置文件结构复杂且缺乏图形化交互，直接使用 HELIOS++ 进行航路规划与参数调试对非专业用户而言存在较大困难。HELIOS++ 官方团队已开发了基于 QGIS 的图形化插件 AEOS，将 HELIOS++ 的仿真配置与执行集成到 QGIS 桌面环境中，为用户提供了 GIS 工作流内的图形化操作界面。本文提出的 FeHALS 则采用 Web 架构，在浏览器中提供轻量级的航路规划与仿真交互体验，与 AEOS 形成互补。

2.2. Web 三维可视化与 Three.js

Three.js 是一个基于 WebGL (Khronos Group, 2011) 的开源 JavaScript 三维图形库，封装了场景 (Scene)、相机 (Camera)、渲染器 (Renderer) 与网格 (Mesh) 等核心对象，支持 OBJ、GLTF、STL 等多种三维模型格式的加载，并提供了轨道控制 (OrbitControls)、拖拽控制 (DragControls) 与射线拾取 (Raycaster) 等交互组件。借助 Three.js，可在浏览器中构建轻量级的三维可视化场景，实现模型展示、点云渲染与鼠标交互拾取，为航路规划提供直观的操作界面。本系统前端即基于 Three.js 实现三维场景渲染与航点交互。

2.3. FastAPI 与后端异步任务

FastAPI 是一个基于 Python 的高性能 Web 框架，基于类型注解与 Pydantic (Colvin, 2019) 实现自动的数据校验与接口文档生成，并原生支持异步编程与 WebSocket。本系统后端基于 FastAPI 构建

REST API (Fielding, 2000) 与 WebSocket 日志通道, 通过 asyncio 子进程异步调用 HELIOS++ 可执行文件, 实时捕获其标准输出并以 WebSocket 推送到前端, 实现仿真过程的实时监控。

3. 系统需求分析与设计

3.1. 用户需求分析

本系统的目标用户为航测工程师、遥感研究人员以及相关专业师生。其核心需求可归纳为三个方面: 一是直观的航路规划, 用户希望在不编写配置文件的情况下, 通过三维场景中的鼠标交互即可完成航点设定与航线编辑; 二是快速的仿真执行, 用户能够通过图形化界面配置平台与扫描仪参数, 一键调用 HELIOS++ 引擎并实时查看运行状态; 三是便捷的结果可视化, 仿真生成的点云能够自动加载到三维场景中, 并支持按高度或强度着色以及尺寸、透明度的调节。

3.2. 系统总体架构

系统采用浏览器/服务器 (B/S) 架构, 分为前端、后端与仿真引擎三层, 其总体架构如图 2 所示。前端基于 Vue 3 构建, 负责三维场景渲染、航点交互编辑、参数配置与结果展示, 通过 HTTP (Fielding et al., 2022; Thomson and Benfield, 2022; Bishop, 2022) 与 WebSocket 与后端通信; 后端基于 FastAPI, 负责接收前端请求, 生成 HELIOS++ 所需的航迹文件与 XML 配置文件, 调用 HELIOS++ 可执行文件执行仿真, 并解析与返回点云结果; 仿真引擎层为 HELIOS++, 独立于前后端, 由后端以子进程方式调用。各模块间通过明确的 REST API 与 WebSocket 消息格式进行数据交换, 形成“数据导入 → 航路规划 → 仿真配置 → 执行控制 → 结果展示 → 日志诊断”的工作流, 如图 3 所示。

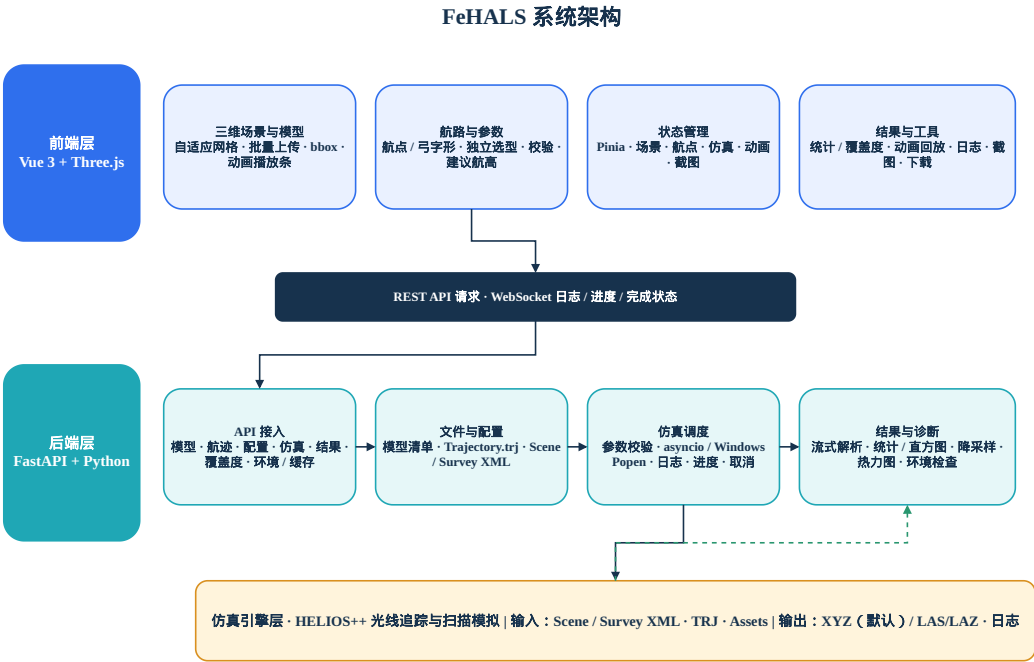


Fig. 2: FeHALS 总体架构

3.3. 功能优先级划分

结合开发环境与工程约束, 系统功能划分为三个优先级: 核心功能 (MVP) 包括三维场景渲染、模型加载、交互式航点编辑、航迹导出、仿真参数配置、仿真执行、日志控制台与点云渲染; 重要功能包括场景要素属性编辑、自动航线生成、航高自动计算、仿真预设方案、点云特征统计与点云下载; 拓展功能包括仿真进度实时反馈、点云覆盖度分析、多任务队列管理与仿真过程动画。本文撰写时仅实现了核心功能, 其余功能仍在开发中, 最新动态请关注 FeHALS github Main Repo。



Fig. 3: FeHALS 数据流

4. 系统设计与实现

4.1. 系统模块划分

系统实现按前后端分层组织，详细的功能划分如图 4 所示。前端基于 Vue 3 与 Node.js (OpenJS Foundation, 2009) 生态构建，目录下包含组件 (components)、状态管理 (stores) 与组合式函数 (composables) 三部分：组件层包含三维场景 Scene3D.vue、参数配置面板 ControlPanel.vue、航点列表 WaypointList.vue 与日志控制台 LogConsole.vue；stores 层基于 Pinia 维护场景 (scene)、航点 (waypoints) 与仿真 (simulation) 三份响应式状态；composables 层封装 Three.js 场景管理 (useThreeScene)、航点交互 (useWaypoints) 与后端接口调用 (useHeliosAPI)。后端基于 FastAPI 构建，api 目录下包含 REST 路由 routes.py 与 WebSocket 端点 websocket.py，models 目录定义 Pydantic 数据模型 schemas.py，services 目录封装航迹生成 trajectory_generator.py、配置生成 config_generator.py、HELIOS++ 调用 helios_service.py 与点云解析 pointcloud_parser.py。

4.2. 三维场景渲染与航点交互

三维场景基于 Three.js 构建，Scene3D.vue 负责初始化场景、相机、渲染器与灯光，并添加地面网格与拾取平面；useThreeScene.js 以单例形式管理渲染循环、模型与点云的增删以及航点的三维表示。场景使用 Z-up 坐标系（与 HELIOS++ 一致，z 为高程轴），通过 OrbitControls 实现相机的旋转、平移与缩放；模型加载采用 OBJLoader、GLTFLoader 与 STLLoader，加载完成后根据模型包围盒自动调整相机视角，并基于模型顶点包围盒推断其 up 轴（Y-up 模型自动旋转至 Z-up）。航点交互通过自绘拖拽实现：鼠标左键点击经 Raycaster 与地面平面求交，在交点处添加球体表示的航点（恒为 $z=0$ 贴地），相邻航点之间以白色折线连接；拖拽航点时锁定相机（‘controls.enabled=false’），确保不干扰视角操作；通过 Delete 键或右键菜单删除航点。

4.3. 前端组件与状态管理

侧边栏采用多 Tab 布局，包含仿真参数、点云、模型列表、航迹与设置五个 Tab。ControlPanel.vue 提供仿真参数表单，分为「载体参数」与「传感器参数」两个模块，各参数旁显示有效范围，只读参数（如光束发散角、最小测程）以灰色禁用显示；参数数据来源于从 HELIOS++ 扫描器与平台目录文件中提取的规格库，切换平台类型时自动重置参数至默认值。WaypointList.vue 以列表形式展示航点序号与坐标，支持单个删除与整体清空。LogConsole.vue 展示分级日志 (INFO/WARNING/ERROR)，支持自动

FeHALS 系统架构图

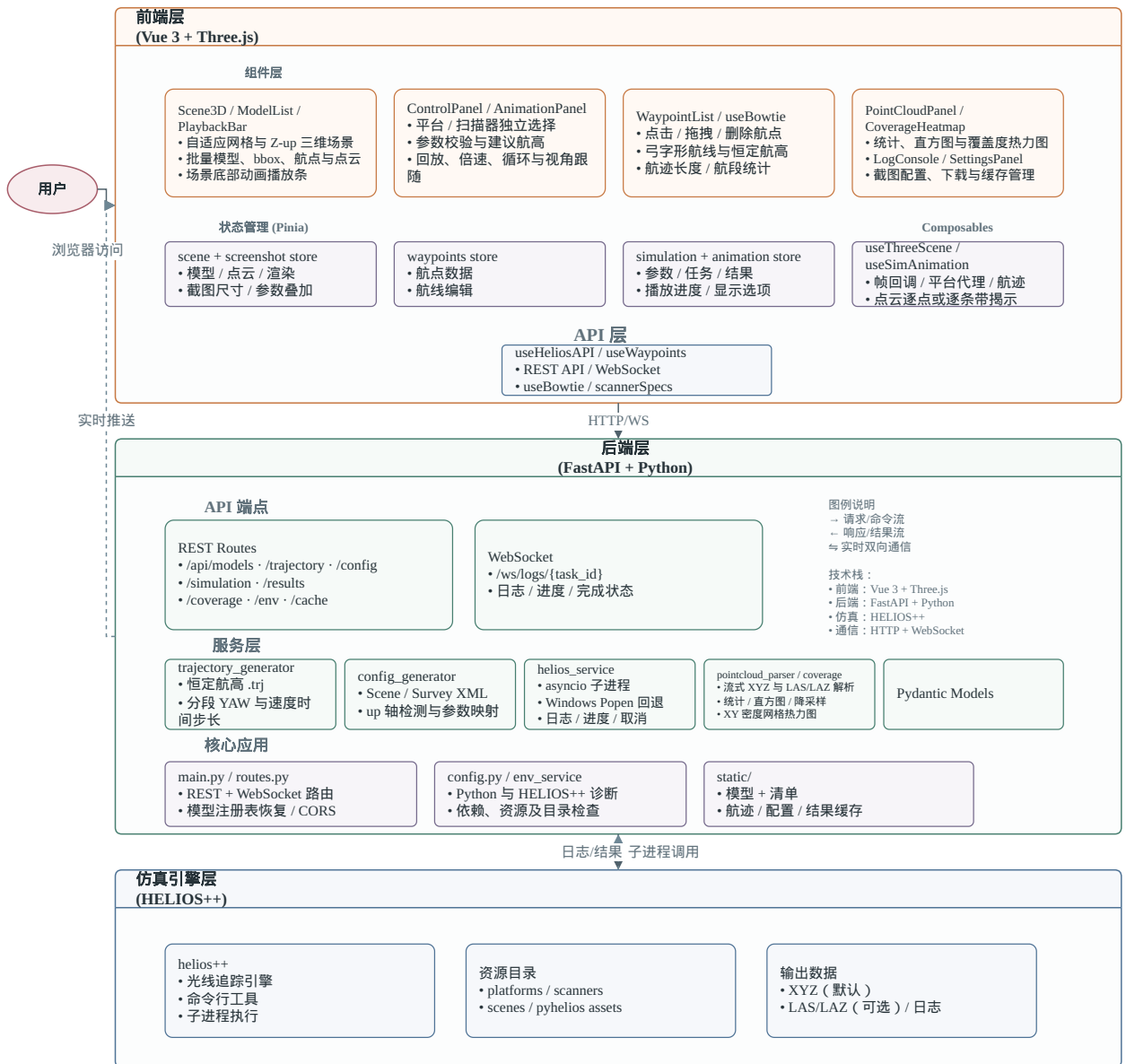


Fig. 4: FeHALS 详细架构

滚动与清空。ModelList.vue 管理已加载模型，支持可见性切换、bbox 线框显示（含尺寸标签）与移除。PointCloudPanel.vue 提供点云渲染参数调节（尺寸、透明度、着色模式）与下载。SettingsPanel.vue 显示各缓存目录的统计信息并支持清理。三个 Pinia store 分别维护场景模型与点云渲染参数（scene）、航点坐标数组（waypoints）以及仿真任务状态、进度、日志与结果（simulation）。前端通过 useHeliosAPI.js 中的 Axios 实例调用后端 REST 接口，并通过 connectLogWS 建立 WebSocket 连接以接收仿真日志。

4.4. 后端服务与接口

后端基于 FastAPI 实现，routes.py 提供模型（上传/列表/删除）、航迹（生成/列表/下载）、配置（生成）、仿真（执行/状态/日志/取消）与结果（元数据/下载）五类 REST 接口，请求与响应由

schemas.py 中的 Pydantic 模型校验；上传的模型与生成的航迹、配置、结果统一存放于静态目录并由 FastAPI 静态文件服务对外提供访问。websocket.py 实现/ws/logs/{task_id} 端点，连接后先回放已有日志，任务运行期间持续推送增量日志与进度消息。main.py 完成 CORS 中间件、静态目录挂载与路由装配。

4.5. 航迹与配置文件生成

trajectory_generator.py 将航点数组按添加顺序、以 1 秒时间间隔生成 HELIOS++ 原生航迹文件 (.trj, CSV 格式，列顺序为时间、roll、pitch、yaw、x、y、z)，所有航点统一为恒定飞行高度（仅用 x、y 定义水平路径）。系统还支持弓字形自动航迹生成：用户在三维场景中点击两个角点定义矩形区域，系统根据设定航线间距自动生成往返覆盖的全覆盖航线。config_generator.py 负责生成 HELIOS++ 所需的两个 XML 文件：场景文件通过 objloader 滤镜引用三维模型，并指定模型的 up 轴（Y-up 自动旋转至 Z-up）；扫描任务（survey）文件引用场景、平台与扫描器目录，并按平台类型生成相应的 survey 结构——机载平台采用插值轨迹（platform=interpolated），车载平台逐航点生成贴地 leg（onGround + movePerSec），地基静态平台使用固定坐标并限定扫描时长（maxDuration_s）；对 360° 旋转 LiDAR（Velodyne VLP-16 / HDL-64E）则将其扫描频率（即转头转速）换算为 headRotatePerSec 参数，将脉冲频率（kHz）与扫描角（± 半角）换算为 HELIOS++ 所需的脉冲频率（Hz）与总扫描角。

4.6. HELIOS++ 调用与点云渲染

helios_service.py 维护仿真任务注册表，通过 asyncio.create_subprocess_exec 以子进程方式调用 helios++ 可执行文件，指定资源目录（-assets）、输出目录（-output）与输出格式参数，实时捕获其标准输出/错误流并解析进度百分比，经 WebSocket 推送至前端，同时支持超时终止与任务取消。仿真启动前，前端对全部参数进行范围校验（数据来源于从 HELIOS++ 扫描器目录文件中提取的参数规格库），超出有效范围时拒绝执行并给出提示。仿真结束后，pointcloud_parser.py 利用 laspy (Montaigu, 2017) 解析 LAS/LAZ 点云或按文本方式解析 XYZ 点云，提取坐标与强度、计算空间范围，并对超过百万级的点云进行降采样。前端将返回的点云坐标构建为 BufferGeometry 与 Points 对象，支持按高度（蓝色至红色渐变）、按强度或固定颜色着色，并可实时调节点尺寸与透明度 (Schütz et al., 2020)。需要说明的是，仅 OBJ 格式模型参与 HELIOS++ 仿真，GLTF/STL 仅用于三维展示；由于本机 HELIOS++ 构建的 LAS 输出不可用，系统默认采用 XYZ 输出格式，详细的数据流见 Fig. 5。

5. 结论与展望

本文针对 HELIOS++ 原生命令行交互复杂、缺乏三维可视化航路规划能力的问题，设计并实现了一个基于 Web 的三维可视化航路规划与激光仿真系统 FeHALS。系统采用 Vue 3 + Three.js 前端与 FastAPI 后端，实现了三维场景渲染、模型加载、交互式航点编辑、弓字形自动航迹生成、航迹文件导出、仿真参数图形化配置（含参数规格库与范围校验）、HELIOS++ 引擎调用、日志实时推送、点云渲染与导出、模型列表管理（可见性/bbox/移除）、缓存清理等核心功能。系统最初面向机载激光扫描（ALS）设计，现已扩展支持车载激光扫描（MLS）与地基激光扫描（TLS）平台，可分别按插值航迹、逐航点贴地运动与固定坐标扫描三种方式生成对应的 survey 配置，并针对 360° 旋转 LiDAR 自动换算转头转速参数。通过机载、车载与地基等典型场景验证了系统的可行性与实用性，有效降低了 HELIOS++ 的使用门槛。

在项目管理方面，本文作者作为仓库唯一的 CODEOWNER，统筹了 10 名成员的协作开发，累计完成 40 余次提交与 8 次 Pull Request 合并。从技术选型决策（摒弃 QGIS 路线转向 Web 架构）、开发规范制定（Git 工作流、分支策略、CODEOWNERS 保护规则）、核心代码实现（三维场景、航点交互、航迹生成、HELIOS++ 集成、点云解析等模块），到文档编写、团队进度管理、代码审查与质量把控，均由作者主导推进。各成员的具体贡献类型详见首页的贡献声明与 Appendix A，详细的提交历史与代码变更可访问 FeHALS github Main Repo。

后续工作将在以下方面展开：（1）实现 HELIOS++ 运行环境自动检测与诊断，系统启动时检测 helios++ 可执行文件是否可用、资源目录（-assets）是否完整，并在前端设置面板中直观显示检测结果，缺失时提供从源码构建到配置的完整引导提示；（2）提供应用环境一键部署方案，包括 Conda 环境配置文件（environment.yml）与自动安装脚本，实现 Python 依赖、Node.js 依赖与前端构建的自动化，并可扩展 Docker 容器化部署方式；（3）引入航高自动计算，根据场景中模型最高点自动推荐安全

FeHALS 数据流程图

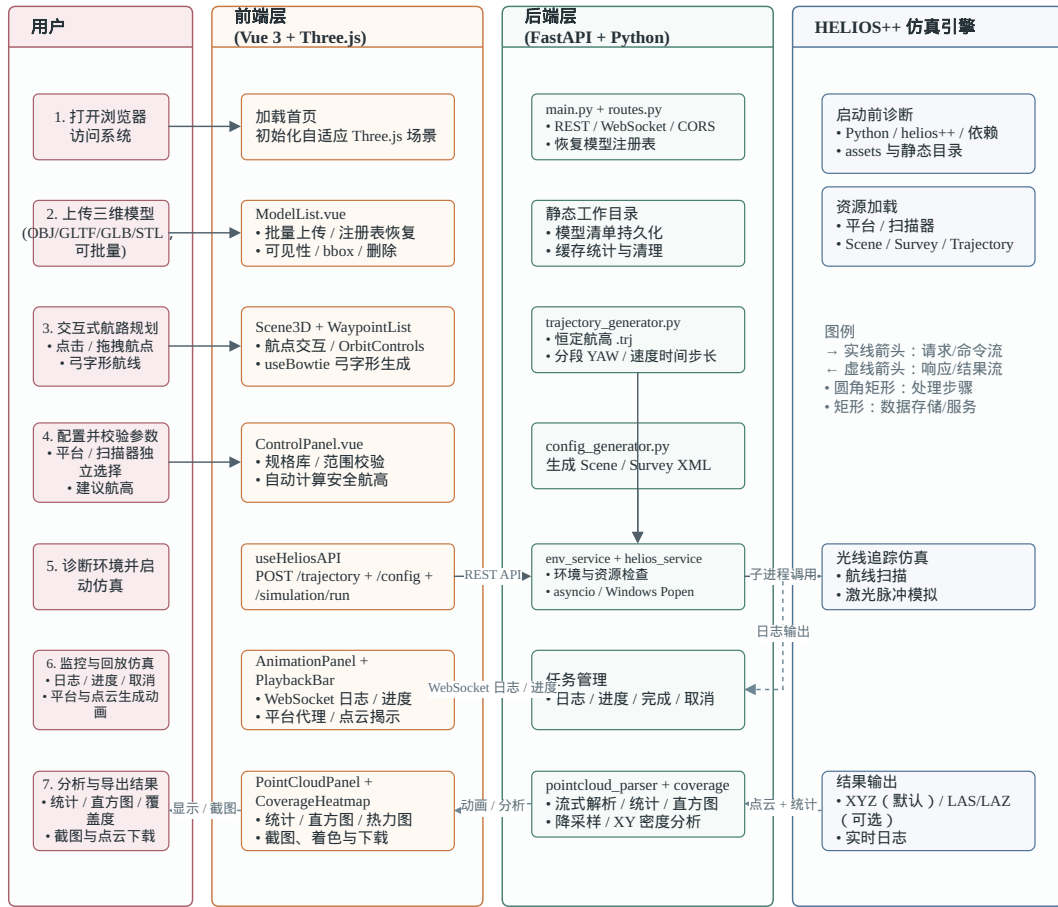


Fig. 5: FeHALS 详细数据流

飞行高度；(4) 实现仿真任务队列管理，支持多任务顺序执行与并发调度，显示任务列表与状态；(5) 增强点云后处理能力，提供覆盖度分析、密度热力图与特征统计等分析功能；(6) 优化大规模点云的渲染性能，采用分层细节 (LOD) 与八叉树等结构支撑千万级点云的流畅交互；(7) 在已支持机载、车载与地基平台的基础上，进一步拓展扫描器与平台规格库，支持用户自定义参数配置，并增加手持式激光雷达等新型平台；(8) 实现仿真过程动画，逐点或逐线显示点云生成过程；(9) 完善车载与手持式激光雷达的路线规划能力，支持根据车辆行驶路线或行人前进路线模拟室内外场景的车载与手持式激光雷达数据采集，进一步提升 FeHALS 作为 HELIOS++ 接口工具在合成数据生成中的应用潜力。

CRedit authorship contribution statement

王孜悠: 文档编写, 思路构建, 代码实现 (FeHALS 最小可用的基本框架), 功能验证, 项目管理, 代码审查, 图表绘制. 温璞: 代码实现 (仿真回放), 可视化. 梅康凯: 代码实现 (航高自动计算与 Windows 启动器), 系统适配. 唐如意: 代码实现 (点云基本特征统计与大范围点云解析性能提升). 杨杰瑞: 文档编写, 图表绘制. 杜仲宇: 文档编写, 图表绘制. 李英杰: 代码实现 (HELIOS++ 仿真引擎可用性监测), 系统适配. 施祺: 代码实现 (点云覆盖度分析). 刘继明: 代码实现 (多任务队列调度). 向宇鸣: 暂无贡献. 隋铭明: 监督指导. 那嘉明: 监督指导. 陈动: 监督指导. 朱杰: 监督指导.

指包含 3D 窗口、模型管理与渲染、航迹规划、仿真参数设置、HELIOS++ 配置生成、HELIOS++ 引擎连接、点云载入与渲染、点云导出、配置导出的可扩展框架。

[illegible]

B. FeHALS 示例 workflow

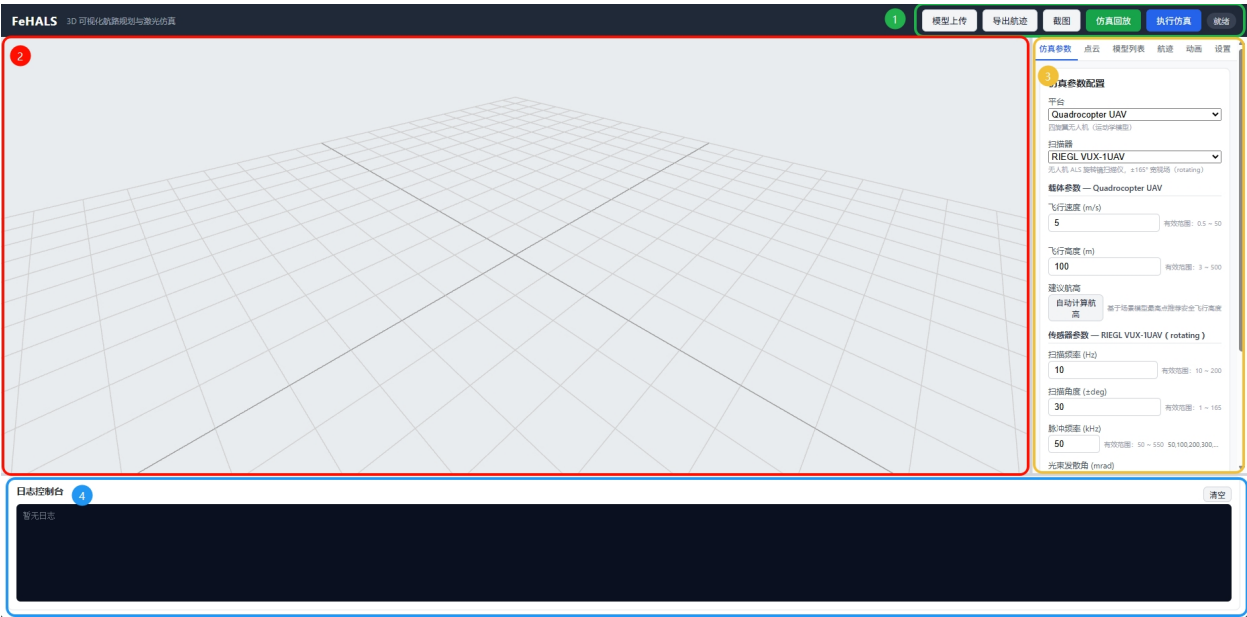


Fig. 7: FeHALS 主页面，其中 1 为工具条，2 为 3D 主视口，3 为菜单栏，4 为日志输出

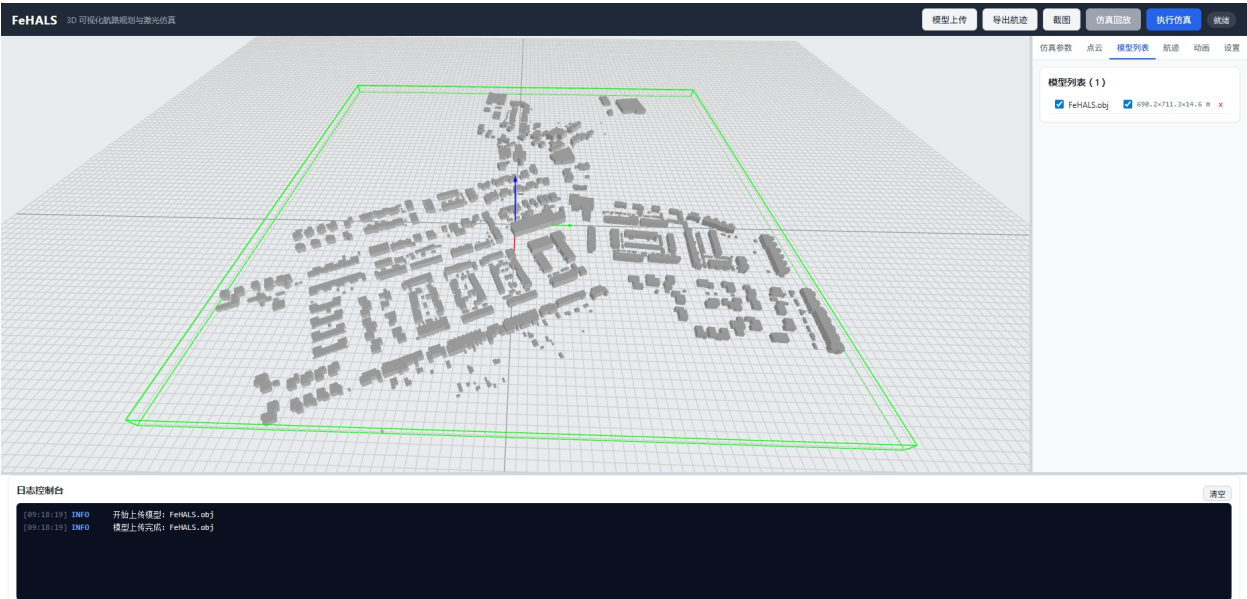


Fig. 8: 模型导入，FeHALS 可以拖动、缩放模型，并显示模型 bbox

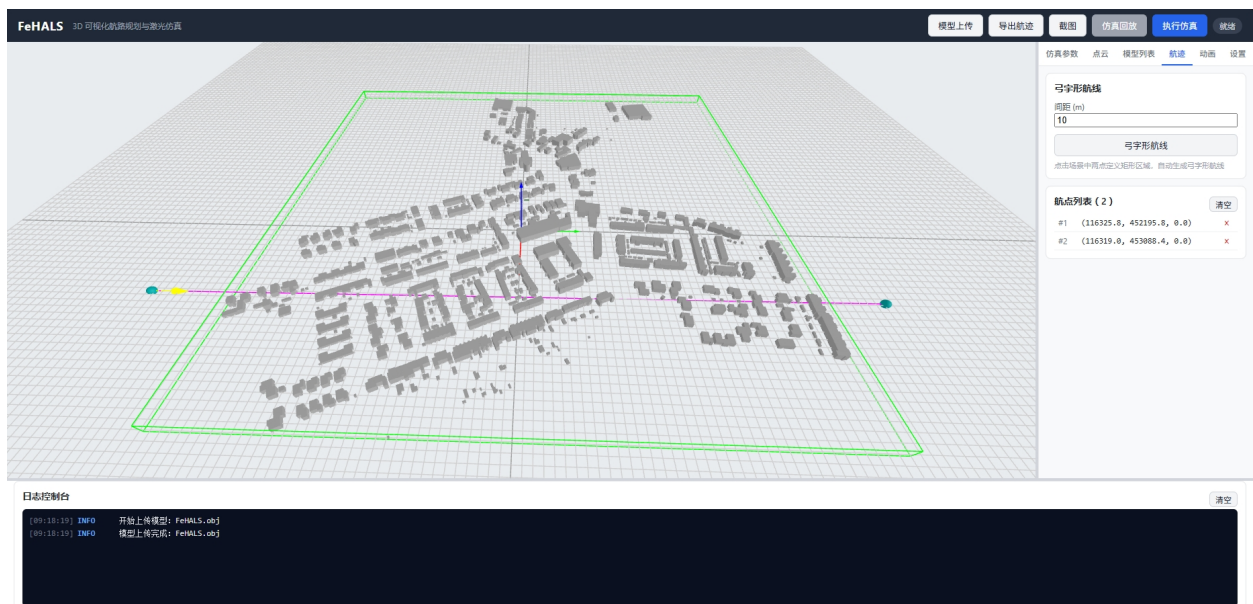


Fig. 9: 航迹绘制

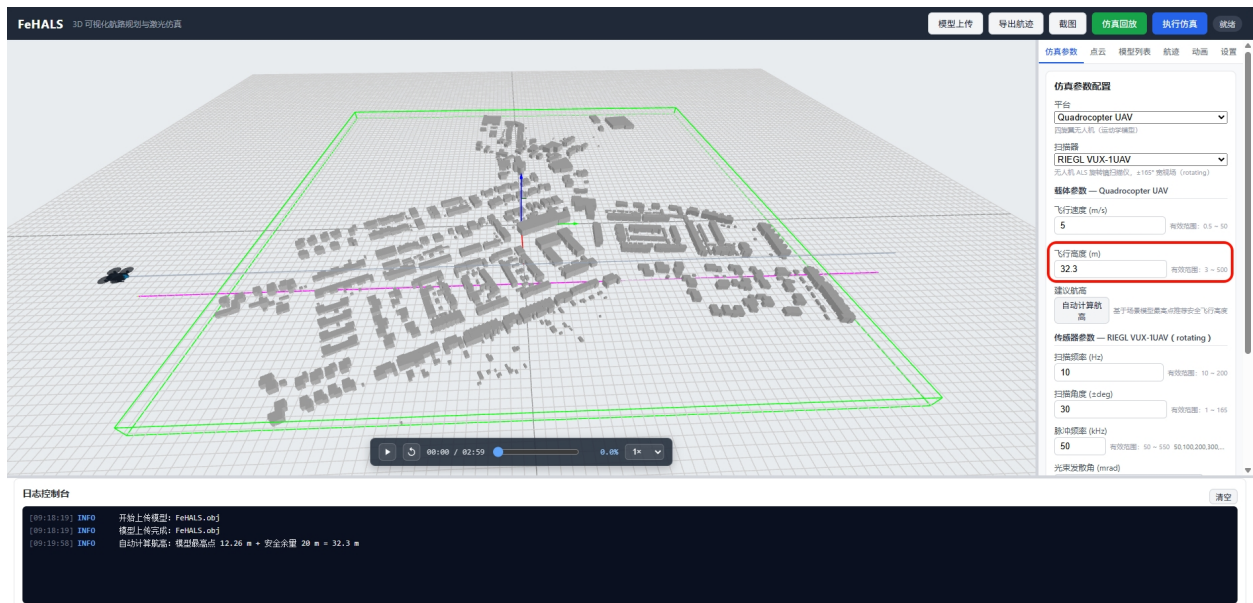
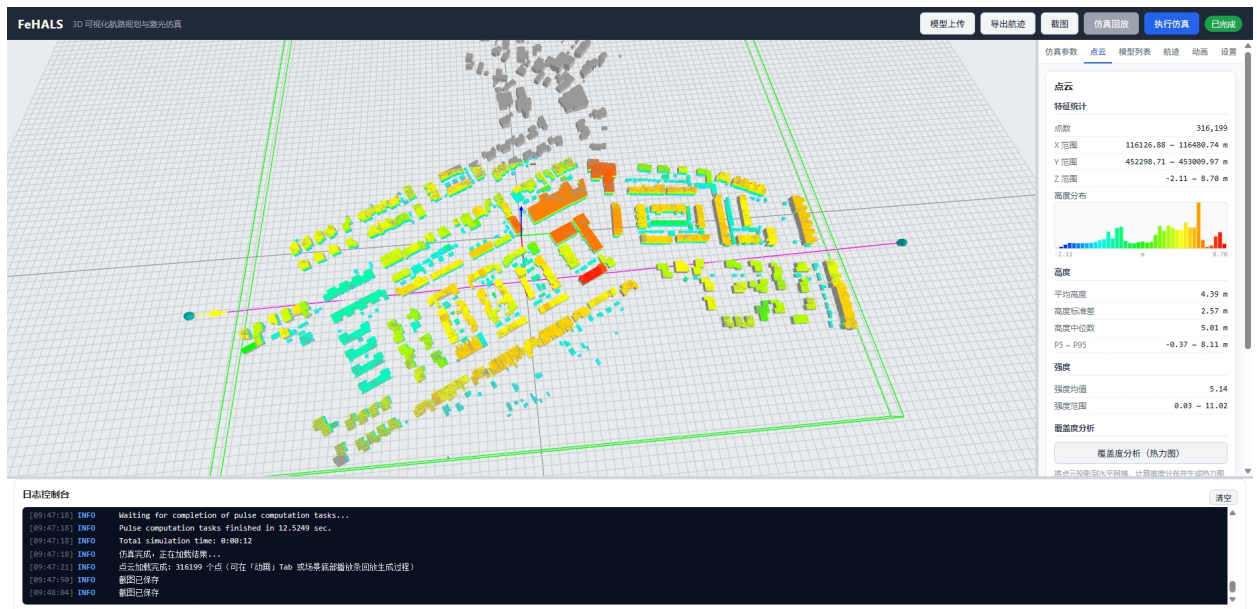


Fig. 10: 仿真参数，主要包含平台和传感器参数

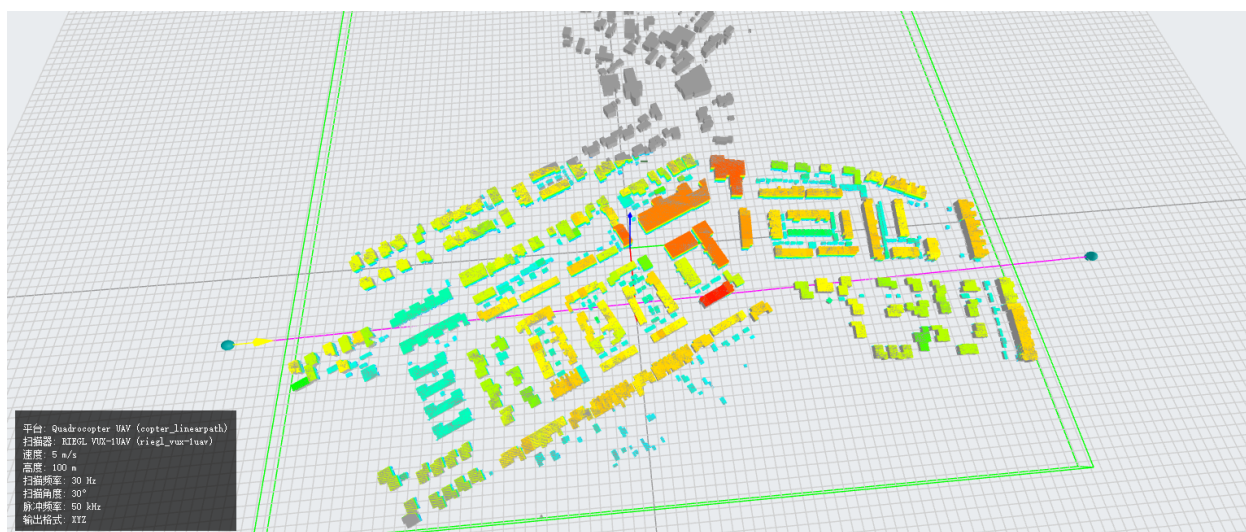


(a) UAV + RIEGL VUX

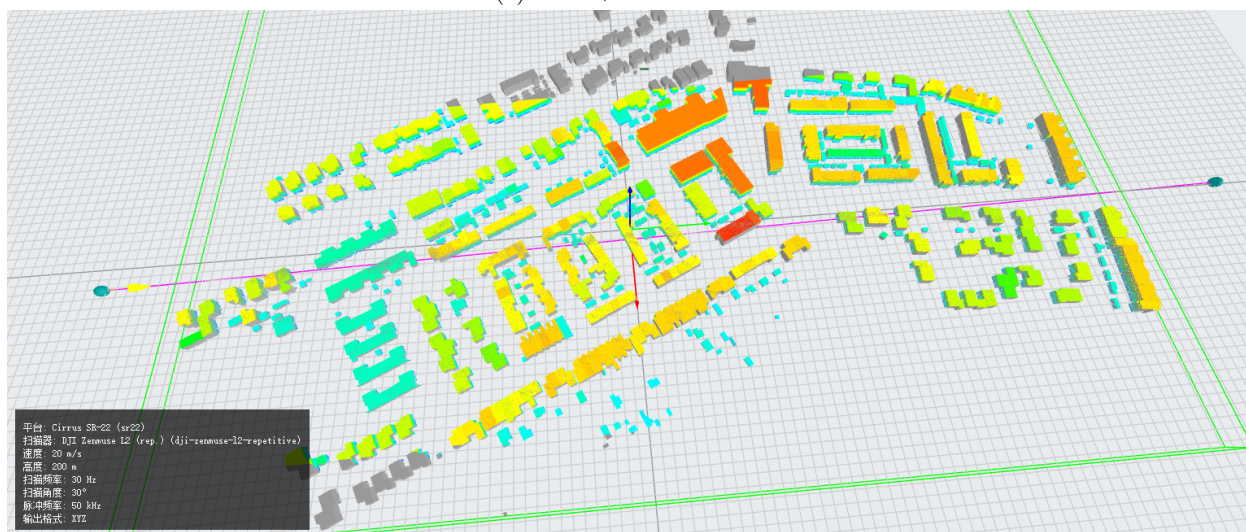


(b) SR22 + DJI ZENMUSE L2

Fig. 11: 不同参数设置的点云总览



(a) UAV + RIEGL VUX



(b) SR22 + DJI ZENMUSE L2

Fig. 12: 不同参数设置的点云细节

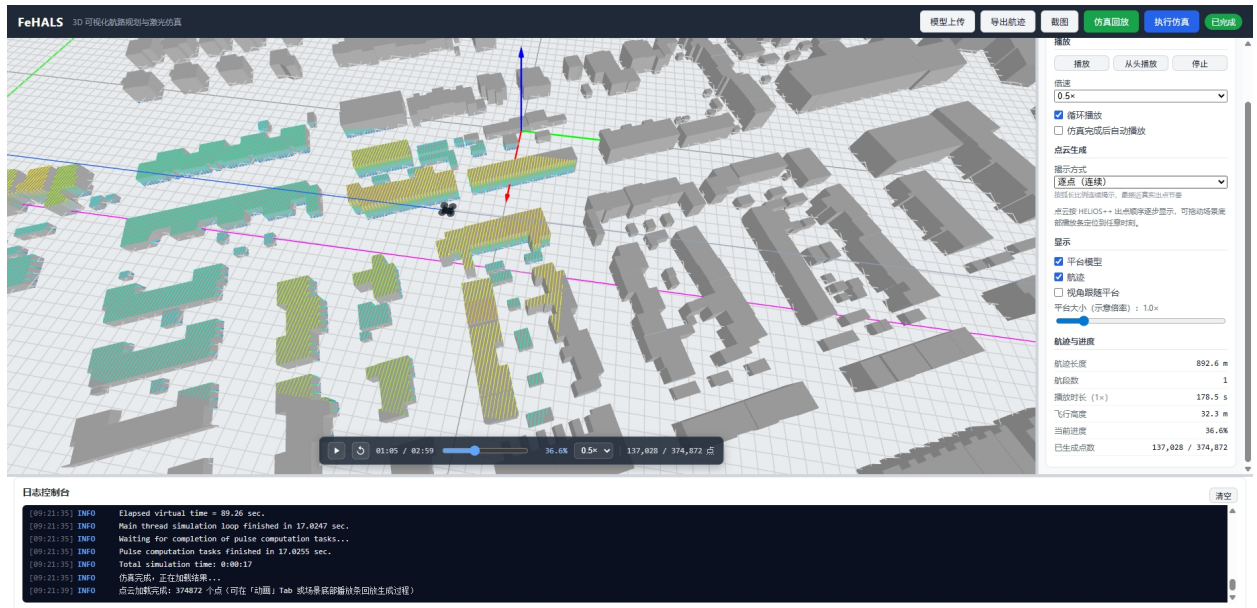


Fig. 13: 逐点仿真回放，也可以选择逐航段

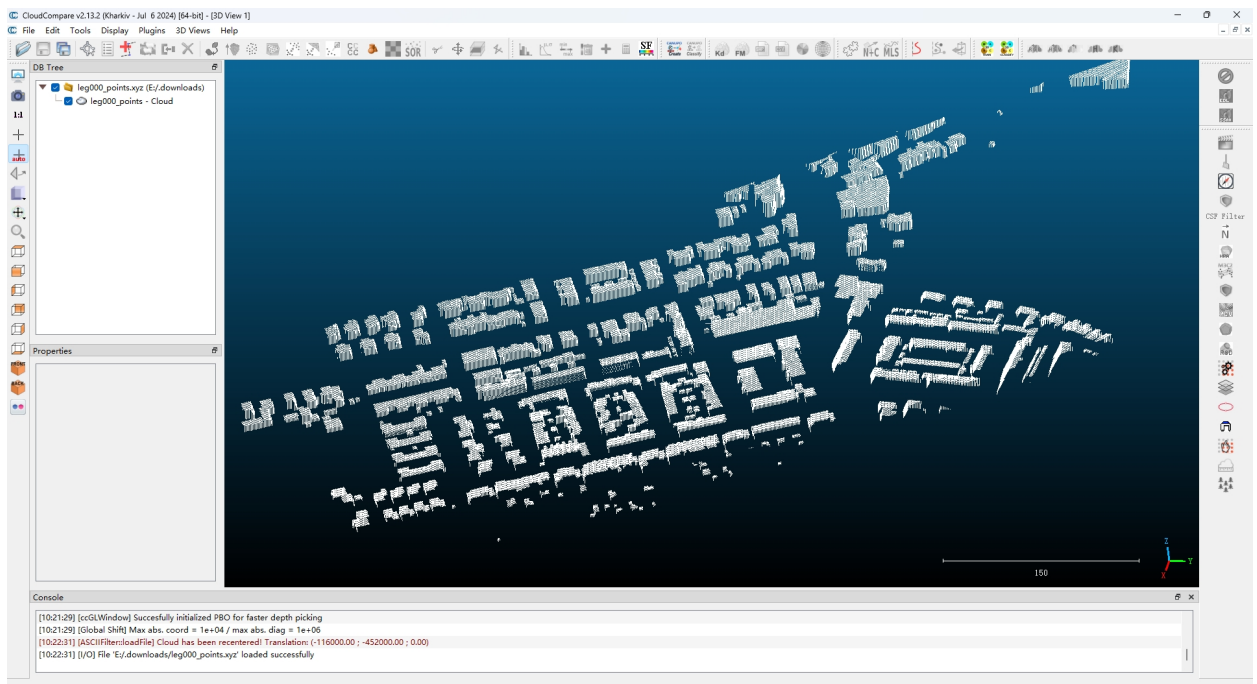
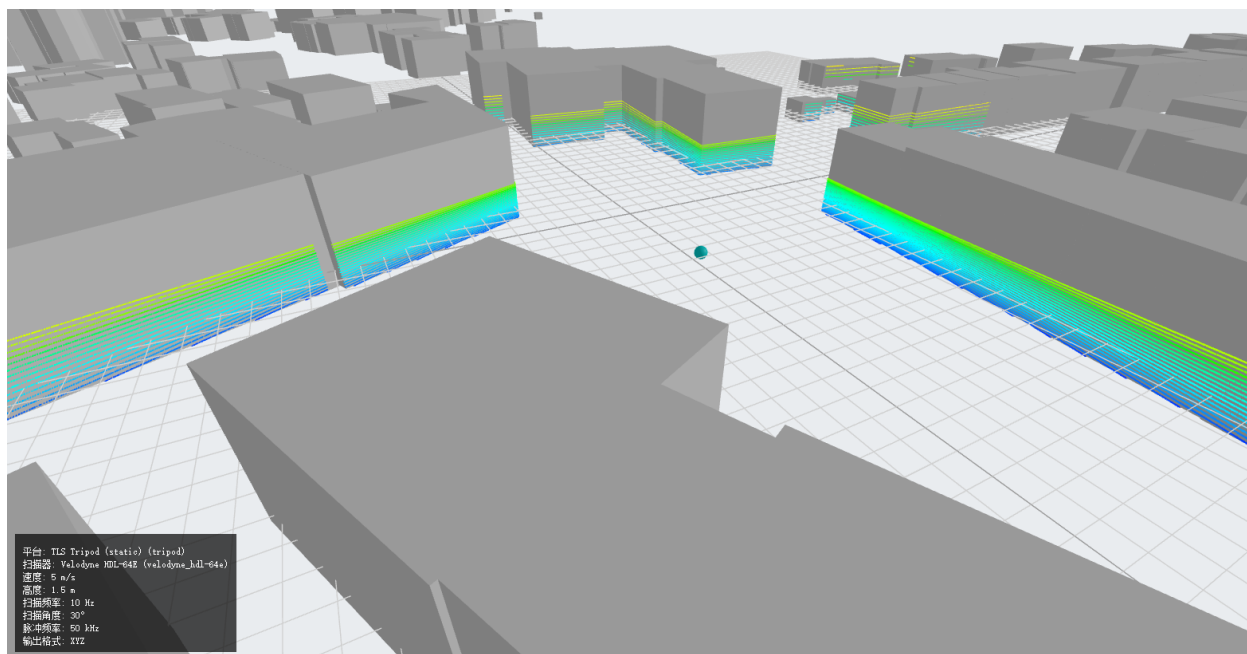
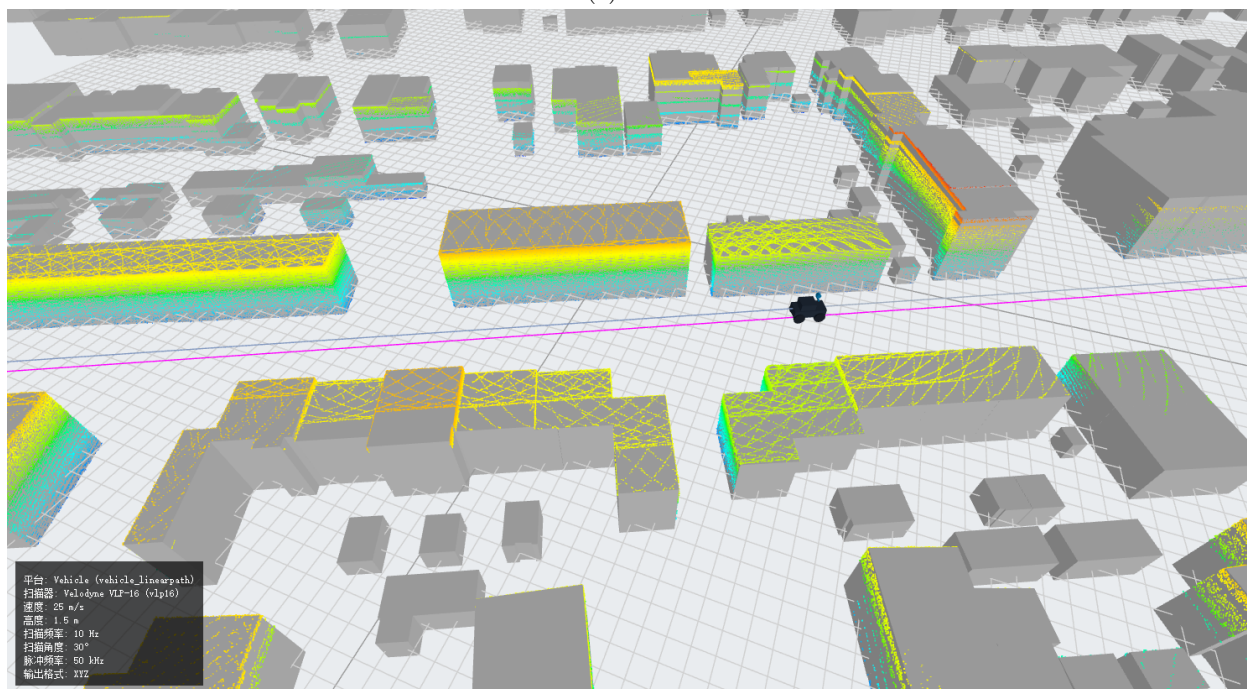


Fig. 14: 导出点云，在 CloudCompare (Girardeau-Montaut, 2011) 中渲染

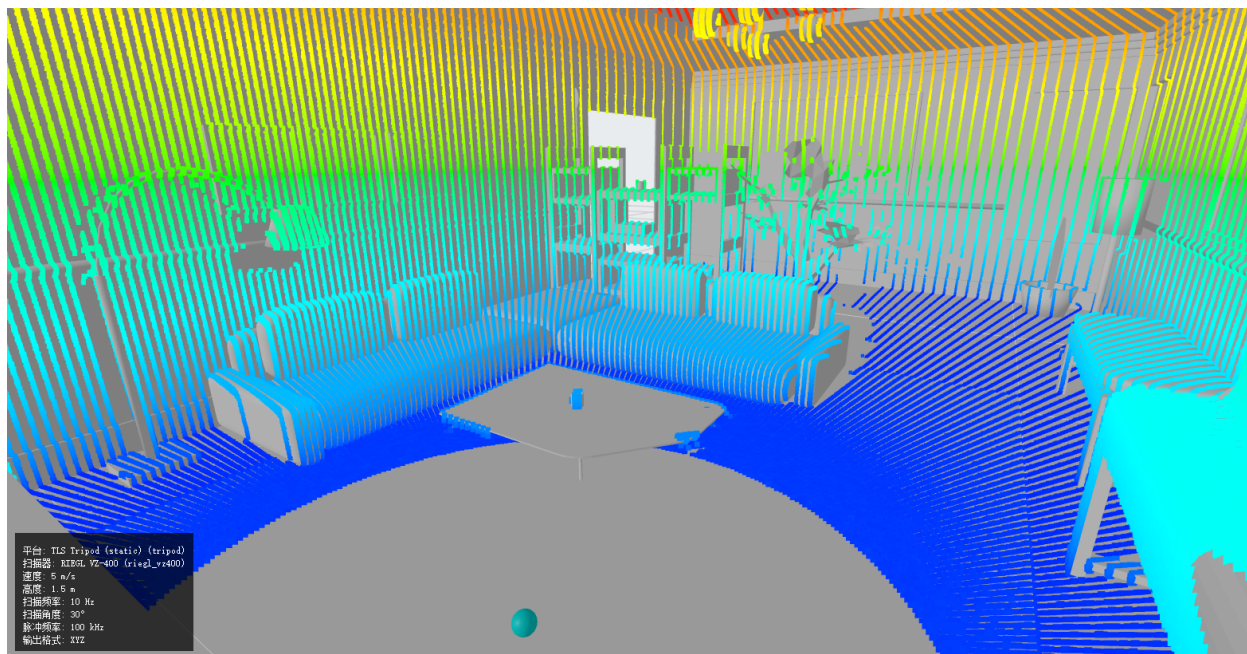


(a) TLS

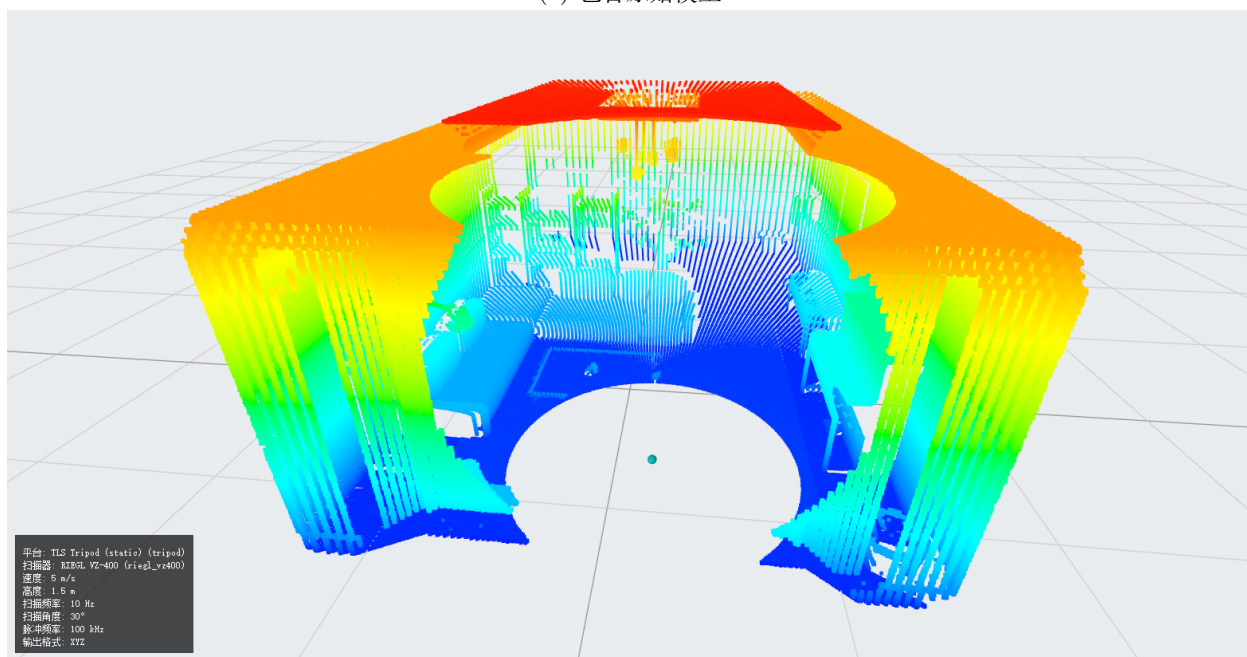


(b) 车载

Fig. 15: 地基平台



(a) 包含原始模型



(b) 除去原始模型

Fig. 16: 室内场景

参考文献

- 3D Geo Research Group, Heidelberg University, 2022. Aeos: A qgis plugin for helios++ lidar simulation. URL: <https://github.com/3dgeo-heidelberg/aeos>.
- Bishop, M., 2022. HTTP/3. RFC 9114. RFC Editor. URL: <https://www.rfc-editor.org/info/rfc9114>, doi:10.17487/RFC9114. proposed Standard.
- Cabello, R., 2010. Three.js: Javascript 3d library. URL: <https://threejs.org/>.
- Colvin, S., 2019. Pydantic: Data validation using python type hints. URL: <https://docs.pydantic.dev/>.
- Fette, I., Melnikov, A., 2011. The WebSocket Protocol. Technical Report 6455. RFC Editor. doi:10.17487/RFC6455.
- Fielding, R., Nottingham, M., Reschke, J., 2022. HTTP Semantics. STD 9110. RFC Editor. URL: <https://www.rfc-editor.org/info/rfc9110>, doi:10.17487/RFC9110. standards Track.
- Fielding, R.T., 2000. Architectural Styles and the Design of Network-based Software Architectures. Ph.D. thesis. University of California, Irvine. URL: <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>. doctoral dissertation.
- Girardeau-Montaut, D., 2011. Cloudcompare: 3d point cloud and mesh processing software. URL: <https://www.cloudcompare.org/>.
- Khronos Group, 2011. WebGL specification. URL: <https://www.khronos.org/registry/webgl/specs/latest/>.
- Montaigu, T., 2017. laspy: Python library for lidar las/laz io. URL: <https://laspy.readthedocs.io/>.
- OpenJS Foundation, 2009. Node.js. URL: <https://nodejs.org/>.
- Ramírez, S., 2018. Fastapi. URL: <https://fastapi.tiangolo.com/>.
- Schütz, M., Kronlachner, M., Wimmer, M., 2020. Progressive real-time rendering of one billion points. Computer Graphics Forum 39, 383–394. doi:10.1111/cgf.13938.
- Thomson, M., Benfield, C., 2022. HTTP/2. RFC 9113. RFC Editor. URL: <https://www.rfc-editor.org/info/rfc9113>, doi:10.17487/RFC9113. proposed Standard.
- Winiwarter, L., Esmoris Pena, A.M., Weiser, H., Anders, K., Martínez Sánchez, J., Searle, M., Höfle, B., 2022. Virtual laser scanning with helios++: A novel take on ray tracing-based simulation of topographic full-waveform 3d laser scanning. Remote Sensing of Environment 269. URL: <https://www.sciencedirect.com/science/article/pii/S0034425721004922>, doi:<https://doi.org/10.1016/j.rse.2021.112772>.
- Yao, W., Li, Y., Shan, J., 2022. Airborne lidar for 3d scene reconstruction: A review. IEEE Geoscience and Remote Sensing Magazine 10, 167–191. doi:10.1109/MGRS.2022.3187698.
- You, E., 2014. Vue.js: The progressive javascript framework. URL: <https://vuejs.org/>.